PLUGIN DISTRIBUTION

Create and distribute a plugin marketplace

Build and host plugin marketplaces to distribute Claude Code extensions across teams and communities.

A **plugin marketplace** is a catalog that lets you distribute plugins to others. Marketplaces provide centralized discovery, version tracking, automatic updates, and support for multiple source types, including git repositories and local paths. This guide shows you how to create your own marketplace to share plugins with your team or community.

Looking to install plugins from an existing marketplace? See **Discover and install prebuilt plugins**.

Overview

Creating and distributing a marketplace involves:

1. **Create plugins**: build one or more plugins with skills, agents, hooks, MCP servers, or LSP servers. This guide assumes you already have plugins to distribute; see **Create plugins** for details on how to create them.
2. **Create the marketplace file**: define a `marketplace.json` that lists your plugins and where to find them. See **Create the marketplace file**.
3. **Host the marketplace**: push to GitHub, GitLab, or another git host. See **Host and distribute marketplaces**.
4. **Share with users**: users add your marketplace with `/plugin marketplace add` and install individual plugins. See **Discover and install plugins**.

Once your marketplace is live, you can update it by pushing changes to your repository. Users refresh their local copy with `/plugin marketplace update`.

Walkthrough: create a local marketplace

This example creates a marketplace with one plugin: a `quality-review` skill for code reviews. You'll

create the directory structure, add a skill, create the plugin manifest and marketplace catalog, then install and test it.

1 Create the directory structure

```
mkdir -p my-marketplace/.claude-plugin
mkdir -p my-marketplace/plugins/quality-review-
plugin/.claude-plugin
mkdir -p my-marketplace/plugins/quality-review-
plugin/skills/quality-review
```

2 Create the skill

Create a `SKILL.md` file that defines what the `quality-review` skill does.

my-marketplace/plugins/quality-review-plugin/skills/quality-review/SKILL.md

```
---
description: Review code for bugs, security, and performance
disable-model-invocation: true
---

Review the code I've selected or the recent changes for:
- Potential bugs or edge cases
- Security concerns
- Performance issues
- Readability improvements

Be concise and actionable.
```

3 Create the plugin manifest

Create a `plugin.json` file that describes the plugin. The manifest goes in the `.claude-plugin/` directory.

my-marketplace/plugins/quality-review-plugin/.claude-plugin/plugin.json

```
{
  "name": "quality-review-plugin",
  "description": "Adds a quality-review skill for quick code reviews",
  "version": "1.0.0"
}
```

Setting `version` means users only receive updates when you change this field, so bump it on every release. If you omit `version` and host this marketplace in git, every commit automatically counts as a new version. See **Version resolution** to choose the right approach.

4 Create the marketplace file

Create the marketplace catalog that lists your plugin.

my-marketplace/.claude-plugin/marketplace.json

```
{
  "name": "my-plugins",
  "owner": {
    "name": "Your Name"
  },
  "plugins": [
    {
      "name": "quality-review-plugin",
      "source": "./plugins/quality-review-plugin",
      "description": "Adds a quality-review skill for quick code reviews"
    }
  ]
}
```

5 **Add and install**

Add the marketplace and install the plugin.

```
/plugin marketplace add ./my-marketplace
/plugin install quality-review-plugin@my-plugins
```

6 **Try it out**

Select some code in your editor and run your new skill. Plugin skills are namespaced with the plugin name.

```
/quality-review-plugin:quality-review
```

To learn more about what plugins can do, including hooks, agents, MCP servers, and LSP servers, see **Plugins**.

> **How plugins are installed**: when users install a plugin, Claude Code copies the plugin directory to a cache location. This means plugins can't reference files outside their directory using paths like `../shared-utils`, because those files won't be copied.
>
> If you need to share files across plugins, use symlinks. See **Plugin caching and file resolution** for details.

Create the marketplace file

Create `.claude-plugin/marketplace.json` in your repository root. This file defines your marketplace's name, owner information, and a list of plugins with their sources.

Each plugin entry needs at minimum a `name` and a `source` that tells Claude Code where to fetch it from. See the **full schema** below for all available fields.

```
{
  "name": "company-tools",
  "owner": {
    "name": "DevTools Team",
    "email": "devtools@example.com"
  },
  "plugins": [
    {
      "name": "code-formatter",
      "source": "./plugins/formatter",
      "description": "Automatic code formatting on save",
      "version": "2.1.0",
      "author": {
        "name": "DevTools Team"
      }
    },
    {
      "name": "deployment-tools",
      "source": {
        "source": "github",
        "repo": "company/deploy-plugin"
      },
      "description": "Deployment automation tools"
    }
  ]
}
```

Marketplace schema

Required fields

Field	Type	Description	Example
`name`	string	Marketplace identifier (kebab-case, no spaces). This is public-facing: users see it when installing plugins (for example, `/plugin install my-tool@your-marketplace`). Each user can register only one marketplace per name: adding a second marketplace with the same name replaces the first. To publish multiple plugins under one marketplace name, list them all in a **single** `marketplace.json`.	`"acme-tools"`

Field	Type	Description	Example
`owner`	object	Marketplace maintainer information (**see fields below**)	
`plugins`	array	List of available plugins	See below

Reserved names: The following marketplace names are reserved for official Anthropic use and cannot be used by third-party marketplaces: `claude-code-marketplace`, `claude-code-plugins`, `claude-plugins-official`, `claude-plugins-community`, `claude-community`, `anthropic-marketplace`, `anthropic-plugins`, `agent-skills`, `anthropic-agent-skills`, `knowledge-work-plugins`, `life-sciences`, `claude-for-legal`, `claude-for-financial-services`, `financial-services-plugins`. Names that impersonate official marketplaces, such as `official-claude-plugins` or `anthropic-tools-v2`, are also blocked.

Owner fields

Field	Type	Required	Description
`name`	string	Yes	Name of the maintainer or team
`email`	string	No	Contact email for the maintainer

Optional fields

Field	Type	Description
`$schema`	string	JSON Schema URL for editor autocomplete and validation. Claude Code ignores this field at load time.
`description`	string	Brief marketplace description
`version`	string	Marketplace manifest version
`metadata.pluginRoot`	string	Base directory prepended to relative plugin source paths (for example, `"./plugins"` lets you write `"source": "formatter"` instead of `"source": "./plugins/formatter"`)
`allowCrossMarketplaceDependenciesOn`	array	Other marketplaces that plugins in this marketplace may depend on. Dependencies from a marketplace not listed here are blocked at install. See **Depend on a plugin from another marketplace**.

`description` and `version` are also accepted under `metadata` for backward compatibility.

Plugin entries

Each plugin entry in the `plugins` array describes a plugin and where to find it. You can include any field from the **plugin manifest schema**, such as `description`, `version`, `author`, `commands`, and `hooks`, plus these marketplace-specific fields: `source`, `category`, `tags`, `strict`, and `relevance`.

Required fields

Field	Type	Description
`name`	string	Plugin identifier (kebab-case, no spaces). This is public-facing: users see it when installing (for example, `/plugin install my-plugin@marketplace`).
`source`	string|object	Where to fetch the plugin from (see **Plugin sources** below)

Optional plugin fields

Standard metadata fields:

Field	Type	Description
`displayName`	string	Human-readable name shown in UI surfaces. Falls back to `name` when omitted. May contain spaces and any casing. Not used for namespacing or lookup. Requires Claude Code v2.1.143 or later.
`description`	string	Brief plugin description
`version`	string	Plugin version. If set (here or in `plugin.json`), the plugin is pinned to this string and users only receive updates when it changes. Omit to fall back to the git commit SHA. See **Version resolution**.
`author`	object	Plugin author information (`name` required, `email` optional)
`homepage`	string	Plugin homepage or documentation URL
`repository`	string	Source code repository URL
`license`	string	SPDX license identifier (for example, MIT, Apache-2.0)
`keywords`	array	Tags for plugin discovery and categorization
`category`	string	Plugin category for organization
`tags`	array	Tags for searchability

Field	Type	Description
`strict`	boolean	Controls whether `plugin.json` is the authority for component definitions (default: true). See **Strict mode** below.
`relevance`	object	Signals that tell Claude Code when to suggest this plugin to users. Takes effect only for marketplaces an administrator allowlists in managed settings. See **Recommend plugins for your org**. Requires Claude Code v2.1.152 or later.
`defaultEnabled`	boolean	Whether the plugin is enabled after install (default: true). Set to `false` to install the plugin disabled until the user opts in. Takes precedence over the same field in the plugin's `plugin.json`. See **Default enablement**. Requires Claude Code v2.1.154 or later.

Component configuration fields:

Field	Type	Description
`skills`	string|array	Custom paths to skill directories containing `<name>/SKILL.md`
`commands`	string|array	Custom paths to flat `.md` skill files or directories
`agents`	string|array	Custom paths to agent files
`hooks`	string|object	Custom hooks configuration or path to hooks file
`mcpServers`	string|object	MCP server configurations or path to MCP config
`lspServers`	string|object	LSP server configurations or path to LSP config

Plugin sources

Plugin sources tell Claude Code where to fetch each individual plugin listed in your marketplace. These are set in the `source` field of each plugin entry in `marketplace.json`.

After Claude Code clones or downloads a plugin to the local machine, it copies the plugin into the local versioned plugin cache at `~/.claude/plugins/cache`.

Source	Type	Fields	Notes
Relative path	`string` (e.g. `"./my-plugin"`)	none	Local directory within the marketplace repo. Must start with `./`. Resolved relative to the marketplace root, not the `.claude-plugin/` directory
`github`	object	`repo`, `ref?`, `sha?`	

Source	Type	Fields	Notes
`url`	object	`url`, `ref?`, `sha?`	Git URL source
`git-subdir`	object	`url`, `path`, `ref?`, `sha?`	Subdirectory within a git repo. Clones sparsely to minimize bandwidth for monorepos
`npm`	object	`package`, `version?`, `registry?`	Installed via `npm install`

Marketplace sources vs plugin sources: These are different concepts that control different things.

- **Marketplace source**: where to fetch the `marketplace.json` catalog itself. Set when users run `/plugin marketplace add` or in `extraKnownMarketplaces` settings. Supports `ref` (branch/tag) but not `sha`.
- **Plugin source**: where to fetch an individual plugin listed in the marketplace. Set in the `source` field of each plugin entry inside `marketplace.json`. Supports both `ref` (branch/tag) and `sha` (exact commit).

For example, a marketplace hosted at `acme-corp/plugin-catalog` (marketplace source) can list a plugin fetched from `acme-corp/code-formatter` (plugin source). The marketplace source and plugin source point to different repositories and are pinned independently.

The git-based source types below are `github`, `url`, and `git-subdir`. When both `ref` and `sha` are set on any of them, the `sha` is the effective pin. Claude Code fetches and checks out the pinned commit directly.

On most git hosts, including GitHub, GitLab, and Bitbucket, this means installation succeeds even if the branch or tag named by `ref` has since been deleted upstream, as long as the commit is still reachable from the repository. Some servers, such as AWS CodeCommit, don't support fetching commits by SHA. On those servers the `ref` must still exist and the pinned commit must be reachable from it.

Relative paths

For plugins in the same repository, use a path starting with `./`:

```
{
  "name": "my-plugin",
  "source": "./plugins/my-plugin"
}
```

Paths resolve relative to the marketplace root, which is the directory containing `.claude-plugin/`. In the example above, `./plugins/my-plugin` points to `<repo>/plugins/my-plugin`, even though `marketplace.json` lives at `<repo>/.claude-plugin/marketplace.json`. Don't use `../` to reference paths outside the marketplace root.

> Relative paths resolve against a local copy of the marketplace, so they work when users add your marketplace from a git source or a local directory. If users add your marketplace via a direct URL to the `marketplace.json` file, relative paths won't resolve, because only that file is downloaded. For URL-based distribution, use GitHub, npm, or git URL sources instead. See **Troubleshooting** for details.

GitHub repositories

```
{
  "name": "github-plugin",
  "source": {
    "source": "github",
    "repo": "owner/plugin-repo"
  }
}
```

You can pin to a specific branch, tag, or commit:

```
{
  "name": "github-plugin",
  "source": {
    "source": "github",
    "repo": "owner/plugin-repo",
    "ref": "v2.0.0",
    "sha": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0"
  }
}
```

Field	Type	Description
`repo`	string	Required. GitHub repository in `owner/repo` format
`ref`	string	Optional. Git branch or tag (defaults to repository default branch)
`sha`	string	Optional. Full 40-character git commit SHA to pin to an exact version

Git repositories

```
{
  "name": "git-plugin",
  "source": {
    "source": "url",
    "url": "https://gitlab.com/team/plugin.git"
  }
}
```

You can pin to a specific branch, tag, or commit:

```
{
  "name": "git-plugin",
  "source": {
    "source": "url",
    "url": "https://gitlab.com/team/plugin.git",
    "ref": "main",
    "sha": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0"
  }
}
```

Field	Type	Description
`url`	string	Required. Full git repository URL (`https://` or `git@`). The `.git` suffix is optional, so Azure DevOps and AWS CodeCommit URLs without the suffix work
`ref`	string	Optional. Git branch or tag (defaults to repository default branch)
`sha`	string	Optional. Full 40-character git commit SHA to pin to an exact version

Git subdirectories

Use `git-subdir` to point to a plugin that lives inside a subdirectory of a git repository. Claude Code uses a sparse, partial clone to fetch only the subdirectory, minimizing bandwidth for large monorepos.

```
{
  "name": "my-plugin",
  "source": {
    "source": "git-subdir",
    "url": "https://github.com/acme-corp/monorepo.git",
    "path": "tools/claude-plugin"
  }
}
```

You can pin to a specific branch, tag, or commit:

```
{
  "name": "my-plugin",
  "source": {
    "source": "git-subdir",
    "url": "https://github.com/acme-corp/monorepo.git",
    "path": "tools/claude-plugin",
    "ref": "v2.0.0",
    "sha": "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0"
  }
}
```

The `url` field also accepts a GitHub shorthand (`owner/repo`) or SSH URLs (`git@github.com:owner/repo.git`).

Field	Type	Description
`url`	string	Required. Git repository URL, GitHub `owner/repo` shorthand, or SSH URL
`path`	string	Required. Subdirectory path within the repo containing the plugin (for example, `"tools/claude-plugin"`)
`ref`	string	Optional. Git branch or tag (defaults to repository default branch)
`sha`	string	Optional. Full 40-character git commit SHA to pin to an exact version

npm packages

Plugins distributed as npm packages are installed using `npm install` . This works with any package on the public npm registry or a private registry your team hosts.

```
{
  "name": "my-npm-plugin",
  "source": {
    "source": "npm",
    "package": "@acme/claude-plugin"
  }
}
```

To pin to a specific version, add the `version` field:

```
{
  "name": "my-npm-plugin",
  "source": {
    "source": "npm",
    "package": "@acme/claude-plugin",
    "version": "2.1.0"
  }
}
```

To install from a private or internal registry, add the `registry` field:

```
{
  "name": "my-npm-plugin",
  "source": {
    "source": "npm",
    "package": "@acme/claude-plugin",
    "version": "^2.0.0",
    "registry": "https://npm.example.com"
  }
}
```

Field	Type	Description
`package`	string	Required. Package name or scoped package (for example, `@org/plugin`)
`version`	string	Optional. Version or version range (for example, `2.1.0`, `^2.0.0`, `~1.5.0`)
`registry`	string	Optional. Custom npm registry URL. Defaults to the system npm registry (typically npmjs.org)

Advanced plugin entries

This example shows a plugin entry using many of the optional fields, including custom paths for commands, agents, hooks, and MCP servers:

```
{
  "name": "enterprise-tools",
  "source": {
    "source": "github",
    "repo": "company/enterprise-plugin"
  },
  "description": "Enterprise workflow automation tools",
  "version": "2.1.0",
  "author": {
    "name": "Enterprise Team",
    "email": "enterprise@example.com"
  },
  "homepage": "https://docs.example.com/plugins/enterprise-
tools",
  "repository": "https://github.com/company/enterprise-plugin",
  "license": "MIT",
  "keywords": ["enterprise", "workflow", "automation"],
  "category": "productivity",
  "commands": [
    "./commands/core/",
    "./commands/enterprise/",
    "./commands/experimental/preview.md"
  ],
  "agents": ["./agents/security-reviewer.md",
"./agents/compliance-checker.md"],
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command":
"${CLAUDE_PLUGIN_ROOT}/scripts/validate.sh"
          }
        ]
      }
    ]
  },
  "mcpServers": {
    "enterprise-db": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
      "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"]
    }
```

```
  },
  "strict": false
}
```

Key things to notice:

- `commands` **and** `agents` : you can specify multiple directories or individual files. Paths are relative to the plugin root.
- `${CLAUDE_PLUGIN_ROOT}` : use this variable in hooks and MCP server configs to reference files within the plugin's installation directory. This is necessary because plugins are copied to a cache location when installed. For dependencies or state that should survive plugin updates, use `${CLAUDE_PLUGIN_DATA}` instead.
- `strict: false` : since this is set to false, the plugin doesn't need its own `plugin.json` . The marketplace entry defines everything. See **Strict mode** below.

By default, a plugin's skills load from the `skills/` directory under its `source` . Paths listed in the `skills` field add to that scan:

```
"skills": ["./skills/", "./extra-skills/"]
```

When several plugin entries share one `skills/` folder at the marketplace root (`source: "./"`), list specific subdirectories instead so each entry loads only its own skills:

```
"source": "./",
"skills": ["./skills/code-review", "./skills/docs"]
```

With a marketplace-root `source` , the listed paths are the complete set for that entry, and other directories in the shared `skills/` folder don't load. Listing `./skills/` itself, or the plugin root, keeps the full scan. If none of the listed paths exist, the default scan runs instead.

Strict mode

The `strict` field controls whether `plugin.json` is the authority for component definitions (skills, agents, hooks, MCP servers, output styles).

Value	Behavior
`true` (default)	`plugin.json` is the authority. The marketplace entry can supplement it with additional components, and both sources are merged.
`false`	The marketplace entry is the entire definition. If the plugin also has a `plugin.json` that declares components, that's a conflict and the plugin fails to load.

When to use each mode:

- `strict: true` : the plugin has its own `plugin.json` and manages its own components. The marketplace entry can add extra skills or hooks on top. This is the default and works for most plugins.
- `strict: false` : the marketplace operator wants full control. The plugin repo provides raw files, and the marketplace entry defines which of those files are exposed as skills, agents, hooks, etc. Useful when the marketplace restructures or curates a plugin's components differently than the plugin author intended.

Host and distribute marketplaces

Host on GitHub (recommended)

GitHub is the recommended way to host and distribute a marketplace:

1. **Create a repository**: set up a new repository for your marketplace
2. **Add marketplace file**: create `.claude-plugin/marketplace.json` with your plugin definitions
3. **Share with teams**: users add your marketplace with `/plugin marketplace add owner/repo`

Benefits: built-in version control, issue tracking, and team collaboration features.

Host on other git services

Any git hosting service works, such as GitLab, Bitbucket, and self-hosted servers. Users add with the full repository URL:

```
/plugin marketplace add https://gitlab.com/company/plugins.git
```

Private repositories

Claude Code supports installing plugins from private repositories. For manual installation and updates, Claude Code uses your existing git credential helpers, so HTTPS access via `gh auth login`, macOS Keychain, or `git-credential-store` works the same as in your terminal. SSH access works as long as the host is already in your `known_hosts` file and the key is loaded in `ssh-agent`, since Claude Code suppresses interactive SSH prompts for the host fingerprint and key passphrase.

Background auto-updates run at startup without credential helpers, since interactive prompts would block Claude Code from starting. To enable auto-updates for private marketplaces, set the appropriate authentication token in your environment:

Provider	Environment variables	Notes
GitHub	`GITHUB_TOKEN` or `GH_TOKEN`	Personal access token or GitHub App token
GitLab	`GITLAB_TOKEN` or `GL_TOKEN`	Personal access token or project token
Bitbucket	`BITBUCKET_TOKEN`	App password or repository access token

Set the token in your shell configuration (for example, `.bashrc`, `.zshrc`) or pass it when running Claude Code:

```
export GITHUB_TOKEN=ghp_xxxxxxxxxxxxxxxxxxxx
```

For CI/CD environments, configure the token as a secret environment variable. GitHub Actions automatically provides `GITHUB_TOKEN` for repositories in the same organization.

Test locally before distribution

Test your marketplace locally before sharing:

```
/plugin marketplace add ./my-local-marketplace
/plugin install test-plugin@my-local-marketplace
```

For the full range of add commands (GitHub, Git URLs, local paths, remote URLs), see **Add marketplaces**.

Require marketplaces for your team

You can configure your repository so team members are automatically prompted to install your marketplace when they trust the project folder. Add your marketplace to `.claude/settings.json`:

```
{
  "extraKnownMarketplaces": {
    "company-tools": {
      "source": {
        "source": "github",
        "repo": "your-org/claude-plugins"
      }
    }
  }
}
```

You can also specify which plugins should be enabled by default:

```
{
  "enabledPlugins": {
    "code-formatter@company-tools": true,
    "deployment-tools@company-tools": true
  }
}
```

For full configuration options, see **Plugin settings**.

If you use a local `directory` or `file` source with a relative path, the path resolves against your repository's main checkout. When you run Claude Code from a git worktree, the path still points at the main checkout, so all worktrees share the same marketplace location. Marketplace state is stored once per user in `~/.claude/plugins/known_marketplaces.json`, not per project.

Pre-populate plugins for containers

For container images and CI environments, you can pre-populate a plugins directory at build time so Claude Code starts with marketplaces and plugins already available, without cloning anything at runtime. Set the `CLAUDE_CODE_PLUGIN_SEED_DIR` environment variable to point at this directory.

To layer multiple seed directories, separate paths with `:` on Unix or `;` on Windows. Claude Code searches each directory in order and uses the first seed that contains a given marketplace or plugin

cache.

The seed directory mirrors the structure of `~/.claude/plugins`:

```
$CLAUDE_CODE_PLUGIN_SEED_DIR/
  known_marketplaces.json
  marketplaces/<name>/...
  cache/<marketplace>/<plugin>/<version>/...
```

To build a seed directory, run Claude Code once during image build, install the plugins you need, then copy the resulting `~/.claude/plugins` directory into your image and point `CLAUDE_CODE_PLUGIN_SEED_DIR` at it.

To skip the copy step, set `CLAUDE_CODE_PLUGIN_CACHE_DIR` to your target seed path during the build so plugins install directly there:

```
CLAUDE_CODE_PLUGIN_CACHE_DIR=/opt/claude-seed claude plugin
marketplace add your-org/plugins
CLAUDE_CODE_PLUGIN_CACHE_DIR=/opt/claude-seed claude plugin
install my-tool@your-plugins
```

Then set `CLAUDE_CODE_PLUGIN_SEED_DIR=/opt/claude-seed` in your container's runtime environment so Claude Code reads from the seed on startup.

At startup, Claude Code registers marketplaces found in the seed's `known_marketplaces.json` into the primary configuration, and uses plugin caches found under `cache/` in place without re-cloning. This works in both interactive mode and non-interactive mode with the `-p` flag.

Behavior details:

- **Read-only**: the seed directory is never written to. Auto-updates are disabled for seed marketplaces since git pull would fail on a read-only filesystem.
- **Seed entries take precedence**: marketplaces declared in the seed overwrite any matching entries in the user's configuration on each startup. To opt out of a seed plugin, use `/plugin disable` rather than removing the marketplace.
- **Path resolution**: Claude Code locates marketplace content by probing `$CLAUDE_CODE_PLUGIN_SEED_DIR/marketplaces/<name>/` at runtime, not by trusting paths stored inside the seed's JSON. This means the seed works correctly even when mounted at a

different path than where it was built.

- **Mutation is blocked**: running `/plugin marketplace remove` or `/plugin marketplace update` against a seed-managed marketplace fails with guidance to ask your administrator to update the seed image.
- **Composes with settings**: if `extraKnownMarketplaces` or `enabledPlugins` declare a marketplace that already exists in the seed, Claude Code uses the seed copy instead of cloning.

Managed marketplace restrictions

For organizations requiring strict control over plugin sources, administrators can restrict which plugin marketplaces users are allowed to add using the `strictKnownMarketplaces` setting in managed settings.

When `strictKnownMarketplaces` is configured in managed settings, the restriction behavior depends on the value:

Value	Behavior
Undefined (default)	No restrictions. Users can add any marketplace
Empty array `[]`	Complete lockdown. Users can't add any new marketplaces
List of sources	Users can only add marketplaces that match the allowlist exactly

Common configurations

Disable all marketplace additions:

```
{
  "strictKnownMarketplaces": []
}
```

Allow specific marketplaces only:

```
{
  "strictKnownMarketplaces": [
    {
      "source": "github",
      "repo": "acme-corp/approved-plugins"
    },
    {
      "source": "github",
      "repo": "acme-corp/security-tools",
      "ref": "v2.0"
    },
    {
      "source": "url",
      "url": "https://plugins.example.com/marketplace.json"
    }
  ]
}
```

Allow all marketplaces from an internal git server using regex pattern matching on the host. This is the recommended approach for **GitHub Enterprise Server** or self-hosted GitLab instances:

```
{
  "strictKnownMarketplaces": [
    {
      "source": "hostPattern",
      "hostPattern": "^github\\.example\\.com$"
    }
  ]
}
```

Allow filesystem-based marketplaces from a specific directory using regex pattern matching on the path:

```
{
  "strictKnownMarketplaces": [
    {
      "source": "pathPattern",
      "pathPattern": "^/opt/approved/"
    }
  ]
}
```

Use `".*"` as the `pathPattern` to allow any filesystem path while still controlling network sources with `hostPattern`.

> `strictKnownMarketplaces` restricts what users can add, but doesn't register marketplaces on its own. To make allowed marketplaces available automatically without users running `/plugin marketplace add`, pair it with `extraKnownMarketplaces` in the same `managed-settings.json`. See **Using both together**.

How restrictions work

Restrictions are checked before any network or filesystem operation. The check runs on marketplace add and on plugin install, update, refresh, and auto-update. If a marketplace was added before the policy was configured and its source no longer matches the allowlist, Claude Code refuses to install or update plugins from it. The same enforcement applies to `blockedMarketplaces`.

The allowlist uses exact matching for most source types. For a marketplace to be allowed, all specified fields must match exactly:

- For GitHub sources: `repo` is required, and `ref` or `path` must also match if specified in the allowlist
- For URL sources: the full URL must match exactly
- For `hostPattern` sources: the marketplace host is matched against the regex pattern
- For `pathPattern` sources: the marketplace's filesystem path is matched against the regex pattern

Exact matching doesn't normalize URLs: a trailing slash, `.git` suffix, or `ssh://` versus `https://` form are treated as different values. If your organization's marketplace can be cloned by more than one URL form, prefer a `hostPattern` entry over a literal URL so all forms match.

Because `strictKnownMarketplaces` is set in **managed settings**, individual users and project

configurations can't override these restrictions.

For complete configuration details including all supported source types and comparison with `extraKnownMarketplaces`, see the **strictKnownMarketplaces reference**.

Version resolution and release channels

Plugin versions determine cache paths and update detection: if the resolved version matches what a user already has, `/plugin update` and auto-update skip the plugin.

Claude Code resolves a plugin's version from the first of these that is set:

1. `version` in the plugin's `plugin.json`
2. `version` in the plugin's marketplace entry
3. The git commit SHA of the plugin's source

For the git-based source types `github`, `url`, `git-subdir`, and relative paths inside a git-hosted marketplace, you can omit `version` entirely and every new commit is treated as a new version. This is the simplest setup for internal or actively-developed plugins.

> ⚠ Setting `version` pins the plugin. If `plugin.json` declares `"version": "1.0.0"`, pushing new commits without changing that string does nothing for existing users, because Claude Code sees the same version and keeps the cached copy. Bump the field on every release, or omit it to use the commit SHA.
>
> Avoid setting `version` in both `plugin.json` and the marketplace entry. Claude Code always uses the `plugin.json` value without warning, so a stale manifest version can mask a version you set in `marketplace.json`.

Set up release channels

To support "stable" and "latest" release channels for your plugins, you can set up two marketplaces that point to different refs or SHAs of the same repo. You can then assign the two marketplaces to different user groups through **managed settings**.

> ⚠ Each channel must resolve to a different version. If you use explicit versions, `plugin.json` must declare a different `version` at each pinned ref. If you omit `version`, the distinct commit SHAs already distinguish the channels. If two refs resolve to the same version string, Claude Code treats them as identical and skips the update.

Example

```
{
  "name": "stable-tools",
  "plugins": [
    {
      "name": "code-formatter",
      "source": {
        "source": "github",
        "repo": "acme-corp/code-formatter",
        "ref": "stable"
      }
    }
  ]
}
```

```
{
  "name": "latest-tools",
  "plugins": [
    {
      "name": "code-formatter",
      "source": {
        "source": "github",
        "repo": "acme-corp/code-formatter",
        "ref": "latest"
      }
    }
  ]
}
```

Assign channels to user groups

Assign each marketplace to the appropriate user group through managed settings. For example, the stable group receives:

```
{
  "extraKnownMarketplaces": {
    "stable-tools": {
      "source": {
        "source": "github",
        "repo": "acme-corp/stable-tools"
      }
    }
  }
}
```

The early-access group receives `latest-tools` instead:

```
{
  "extraKnownMarketplaces": {
    "latest-tools": {
      "source": {
        "source": "github",
        "repo": "acme-corp/latest-tools"
      }
    }
  }
}
```

Pin dependency versions

A plugin can constrain its dependencies to a semver range so that updates to a dependency don't break the dependent plugin. See **Constrain plugin dependency versions** for the `{plugin-name}--v{version}` git-tag convention, range syntax, and how multiple constraints on the same dependency are combined.

Validation and testing

Test your marketplace before sharing.

Validate your marketplace JSON syntax:

```
claude plugin validate .
```

Or from within Claude Code:

```
/plugin validate .
```

Add the marketplace for testing:

```
/plugin marketplace add ./path/to/marketplace
```

Install a test plugin to verify everything works:

```
/plugin install test-plugin@marketplace-name
```

For complete plugin testing workflows, see **Test your plugins locally**. For technical troubleshooting, see **Plugins reference**.

Manage marketplaces from the CLI

Claude Code provides non-interactive `claude plugin marketplace` subcommands for scripting and automation. These are equivalent to the `/plugin marketplace` commands available inside an interactive session.

Plugin marketplace add

Add a marketplace from a GitHub repository, git URL, remote URL, or local path.

```
claude plugin marketplace add <source> [options]
```

Arguments:

- `<source>`: GitHub `owner/repo` shorthand, git URL, remote URL to a `marketplace.json` file, or local directory path. To pin to a branch or tag, append `@ref` to the GitHub shorthand or `#ref` to a git URL

Options:

Option	Description	Default
`--scope <scope>`	Where to declare the marketplace: `user`, `project`, or `local`. See **Plugin installation scopes**	`user`
`--sparse <paths...>`	Limit checkout to specific directories via git sparse-checkout. Useful for monorepos	

Add a marketplace from GitHub using `owner/repo` shorthand:

```
claude plugin marketplace add acme-corp/claude-plugins
```

Pin to a specific branch or tag with `@ref`:

```
claude plugin marketplace add acme-corp/claude-plugins@v2.0
```

Add from a git URL on a non-GitHub host:

```
claude plugin marketplace add
https://gitlab.example.com/team/plugins.git
```

Add from a remote URL that serves the `marketplace.json` file directly:

```
claude plugin marketplace add
https://example.com/marketplace.json
```

Add from a local directory for testing:

```
claude plugin marketplace add ./my-marketplace
```

Declare the marketplace at project scope so it is shared with your team via `.claude/settings.json`:

```
claude plugin marketplace add acme-corp/claude-plugins --scope
project
```

For a monorepo, limit the checkout to the directories that contain plugin content:

```
claude plugin marketplace add acme-corp/monorepo --sparse
.claude-plugin plugins
```

Plugin marketplace list

List all configured marketplaces.

```
claude plugin marketplace list [options]
```

Options:

Option	Description
`--json`	Output as JSON

With `--json`, each entry includes `name`, `source`, and source-specific fields: `repo` for GitHub sources, `url` for git and URL sources, and `path` for local sources. GitHub and git sources also include a `ref` field when the marketplace was added with a pinned branch or tag.

Plugin marketplace remove

Remove a configured marketplace. The alias `rm` is also accepted.

```
claude plugin marketplace remove <name> [options]
```

Arguments:

- `<name>`: marketplace name to remove, as shown by `claude plugin marketplace list`. This is the `name` from `marketplace.json`, not the source you passed to `add`

Options:

Option	Description	Default
`--scope <scope>`	Restrict removal to a single settings scope: `user`, `project`, or `local`. See **Plugin installation scopes**. When omitted, the	(all scopes)

Option	Description	Default
	declaration is removed from every editable scope. When given, only that scope's declaration is removed; the shared state, cache, and installed plugin data are preserved when the marketplace is still declared in another scope	

⚠ Removing a marketplace from its last remaining scope also uninstalls any plugins you installed from it. To refresh a marketplace without losing installed plugins, use `claude plugin marketplace update` instead.

Plugin marketplace update

Refresh marketplaces from their sources to retrieve new plugins and version changes.

```
claude plugin marketplace update [name]
```

Arguments:

- `[name]`: marketplace name to update, as shown by `claude plugin marketplace list`. Updates all marketplaces if omitted

Both `remove` and `update` fail when run against a seed-managed marketplace, which is read-only. When updating all marketplaces, seed-managed entries are skipped and other marketplaces still update. To change seed-provided plugins, ask your administrator to update the seed image. See **Pre-populate plugins for containers**.

Troubleshooting

Marketplace not loading

Symptoms: Can't add marketplace or see plugins from it

Solutions:

- Verify the marketplace URL is accessible
- Check that `.claude-plugin/marketplace.json` exists at the specified path
- Ensure JSON syntax is valid using `claude plugin validate` or `/plugin validate`. To check skill, agent, and command frontmatter, run the command against each plugin directory

- For private repositories, confirm you have access permissions

Marketplace validation errors

Run `claude plugin validate .` or `/plugin validate .` from your marketplace directory to check for issues. When pointed at a marketplace directory, the validator checks `marketplace.json` only: schema, duplicate plugin names, source path traversal, and version mismatches against each referenced `plugin.json`.

To validate an individual plugin's `plugin.json` and its skill, agent, command, and hook files, run the command against the plugin directory itself, for example `claude plugin validate ./plugins/my-plugin`. Common errors:

Error	Cause	Solution
`File not found: .claude-plugin/marketplace.json`	Missing manifest	Create `.claude-plugin/marketplace.json` with required fields
`Invalid JSON syntax: Unexpected token...`	JSON syntax error in marketplace.json	Check for missing commas, extra commas, or unquoted strings
`Duplicate plugin name "x" found in marketplace`	Two plugins share the same name	Give each plugin a unique `name` value
`plugins[0].source: Path contains ".."`	Source path contains `..`	Use paths relative to the marketplace root without `..`. See **Relative paths**
`YAML frontmatter failed to parse: ...`	Invalid YAML in a skill, agent, or command file	Fix the YAML syntax in the frontmatter block. At runtime this file loads with no metadata. Reported only when validating a plugin directory
`Invalid JSON syntax: ...` (hooks.json)	Malformed `hooks/hooks.json`	Fix JSON syntax. A malformed `hooks/hooks.json` prevents the entire plugin from loading. Reported only when validating a plugin directory

Warnings (non-blocking):

- `Marketplace has no plugins defined`: add at least one plugin to the `plugins` array
- `No marketplace description provided`: add a top-level `description` to help users understand your marketplace
- `Plugin name "x" is not kebab-case`: the plugin name contains uppercase letters, spaces, or special characters. Rename to lowercase letters, digits, and hyphens only (for example, `my-plugin`). Claude Code accepts other forms, but the claude.ai marketplace sync rejects them.

Plugin installation failures

Symptoms: Marketplace appears but plugin installation fails

Solutions:

- Verify plugin source URLs are accessible
- Check that plugin directories contain required files
- For GitHub sources, ensure repositories are public or you have access
- Test plugin sources manually by cloning/downloading
- If the source pins both `ref` and `sha`, a deleted upstream branch or tag doesn't block installation on most git hosts, including GitHub, GitLab, and Bitbucket. On servers that don't support fetching commits by SHA, such as AWS CodeCommit, the `ref` must still exist and the pinned commit must be reachable from it. If the install still fails, confirm the pinned commit still exists in the repository

Private repository authentication fails

Symptoms: Authentication errors when installing plugins from private repositories

Solutions:

For manual installation and updates:

- Verify you're authenticated with your git provider (for example, run `gh auth status` for GitHub)
- Check that your credential helper is configured correctly: `git config --global credential.helper`
- Try cloning the repository manually to verify your credentials work

For background auto-updates:

- Set the appropriate token in your environment: `echo $GITHUB_TOKEN`
- Check that the token has the required permissions (read access to the repository)
- For GitHub, ensure the token has the `repo` scope for private repositories
- For GitLab, ensure the token has at least `read_repository` scope
- Verify the token hasn't expired

Marketplace updates fail in offline environments

Symptoms: Marketplace `git pull` fails and Claude Code wipes the existing cache, causing plugins to become unavailable.

Cause: By default, when a `git pull` fails, Claude Code removes the stale clone and attempts to re-clone. In offline or airgapped environments, re-cloning fails the same way, leaving the marketplace directory empty.

Solution: Set `CLAUDE_CODE_PLUGIN_KEEP_MARKETPLACE_ON_FAILURE=1` to keep the existing cache when the pull fails instead of wiping it:

```
export CLAUDE_CODE_PLUGIN_KEEP_MARKETPLACE_ON_FAILURE=1
```

With this variable set, Claude Code retains the stale marketplace clone on `git pull` failure and continues using the last-known-good state. For fully offline deployments where the repository will never be reachable, use `CLAUDE_CODE_PLUGIN_SEED_DIR` to pre-populate the plugins directory at build time instead.

Git operations time out

Symptoms: Plugin installation or marketplace updates fail with a timeout error like “Git clone timed out after 120s” or “Git pull timed out after 120s”.

Cause: Claude Code uses a 120-second timeout for all git operations, including cloning plugin repositories and pulling marketplace updates. Large repositories or slow network connections may exceed this limit.

Solution: Increase the timeout using the `CLAUDE_CODE_PLUGIN_GIT_TIMEOUT_MS` environment variable. The value is in milliseconds:

```
export CLAUDE_CODE_PLUGIN_GIT_TIMEOUT_MS=300000  # 5 minutes
```

Plugins with relative paths fail in URL-based marketplaces

Symptoms: Added a marketplace via URL (such as `https://example.com/marketplace.json`), but plugins with relative path sources like `"./plugins/my-plugin"` fail to install with “path not found” errors.

Cause: URL-based marketplaces only download the `marketplace.json` file itself. They don't download plugin files from the server. Relative paths in the marketplace entry reference files on the remote server that were not downloaded.

Solutions:

- **Use external sources**: Change plugin entries to use GitHub, npm, or git URL sources instead of relative paths:

  ```
  { "name": "my-plugin", "source": { "source": "github", "repo": "owner/repo" } }
  ```

- **Use a Git-based marketplace**: Host your marketplace in a Git repository and add it with the git URL. Git-based marketplaces clone the entire repository, making relative paths work correctly.

Files not found after installation

Symptoms: Plugin installs but references to files fail, especially files outside the plugin directory

Cause: Plugins are copied to a cache directory rather than used in-place. Paths that reference files outside the plugin's directory (such as `../shared-utils`) won't work because those files aren't copied.

Solutions: See **Plugin caching and file resolution** for workarounds including symlinks and directory restructuring.

For additional debugging tools and common issues, see **Debugging and development tools**.

See also

- **Discover and install prebuilt plugins** - Installing plugins from existing marketplaces
- **Plugins** - Creating your own plugins
- **Plugins reference** - Complete technical specifications and schemas
- **Plugin settings** - Plugin configuration options
- **strictKnownMarketplaces reference** - Managed marketplace restrictions